

Вајрцен

Wiregen

Аутор:

ЛУКА ЋИРКОВИЋ

*2. Разред, Електротехничка школа „Земун“, Београд, Регионални центар за таленте
Земун I*

Ментори:

РЕЉА ЋУРЧИН,

Професор, Електротехничка школа „Земун“, Београд

ДИЈАНА СТЕФАНОВИЋ

Професор, Електротехничка школа „Земун“, Београд

РЕЗИМЕ: У овом раду приказујем манипулацију тродимензионалних објеката у C# вижуал студио окружењу. Циљ овог рада јесте да приближи и олакша кориснику креирање комплексних 3Д модела управо користећи мој програм. За реализацију овог рада морао сам да користим математику за калкулисање позиција и ротација тачака у тродимензионалном простору као и перспективну пројекцију, као и познавање основних функција које се могу пронаћи у сличним софтверима за 3Д манипулацију као што је “Blender”¹, “Autodesk Maya”², “SketchUp”³ и сл. и имплементирати их у свој пројекат. За овај пројекат коришћени су стручни радови као и видео упутства ради бољег разумевања мог рада.

КЉУЧНЕ РЕЧИ: 3Д манипулација, модели, креирати, тродимензионални простор

ABSTRACT: In this work I’m presenting three-dimentional object manipulation inside of the C# Visual Studio enviroment. The goal of this work is to bring closer and ease in the user into creating complex 3D models by using my programme. For the realisation of this work I used math for calculating positions and rotations of the points in 3D space and calculating the perspective projection, as well as knowing the basics in similar 3D manipulation softwares that already exist on the market such as “Blender”, “Autodesk Maya”, “SketchUp” and implement them into my project. For Wiregen I used professional papers and video tutorials to better understand the material shown in this work.

KEYWORDS: 3D manipulation, models, creating, three-dimentional space

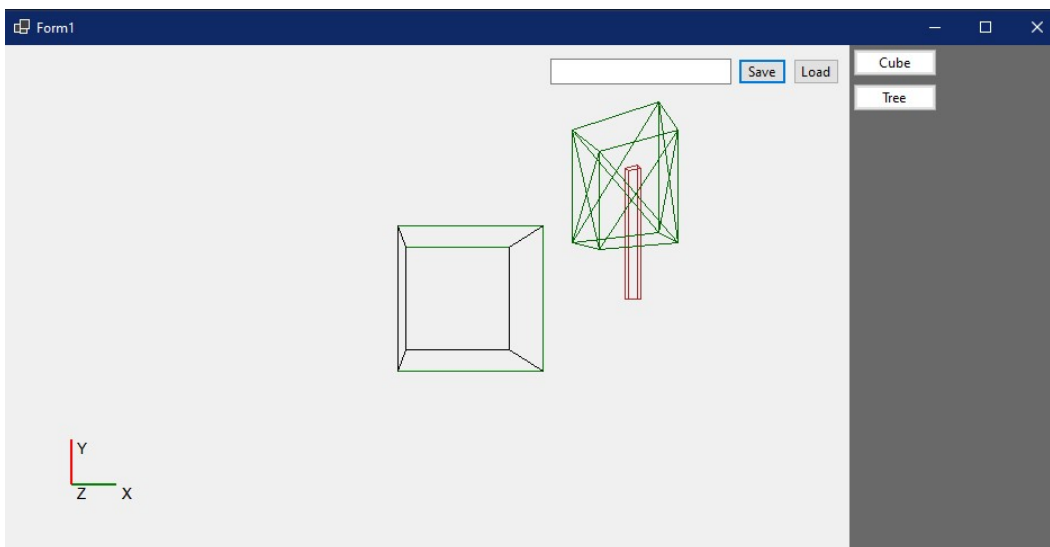
¹ <https://www.blender.org/>

² <https://www.autodesk.com/products/maya/overview>

³ <https://www.sketchup.com/en>

УВОД

Свима који се баве у секторима грађевинарства, као што су архитекте или сасвим обични који хоће да направе нпр.⁴ неки прототип полице или програмери као прототип неких карактера, па чак и за прављење 3Д пејзажа морали би да науче неки од популарних софтвера за управо то. Они могу да буду препуни информација и тиме би нови корисници преоптерећени. Зато сам одлучио да направим програм који је неинтензиван, не троши доста ресурса компјутера и притом дозвољава за креирање комплекснијих модела за кратко време.



Слика 1 – Изглед програма;
Figure 1 – First Look;

Овај истраживачки рад ће детаљно објаснити како 3Д програми раде и како компјутери нама омогућавају перцепцију 3Д елемената. Описаћу материјал и методе које су коришћене за реализацију овог пројекта као и дизајн и резултате свог рада.

⁴ На пример

МЕТОДОЛОГИЈА

Прва идеја за овакву апликацију сам добио још у седмом разреду основне школе. Претходне године сам успео да реализујем апликацију сличну Вајрцену, под именом „Визио3Д”, међутим није постојао лак начин за додавање нових модела, систем је био доста спорији и неоптимизован, а и намена је била за приказивање ротације објекта. Може се рећи да је био претходник Вајрцена.

Али како апликација приказује 3Д објекте? Пре свега, тродимензионални простор јесте математички простор који садржи три координате, X, Y и Z. Те три координате су неопходне за одређивање позиције тачке у том 3Д свету. Након тога, линијама повезујемо тачке које хоћемо. Наравно, морамо да имамо начин како да прикажемо те линије, тј.⁵ у каквој пројекцији хоћемо да видимо те линије у 3Д простору. У стварном свету, људи виде у перспективи са једном тачком. С обзиром да правимо 3Д рендер енцип⁶, и ми ћемо то исто урадити и за мој пројекат.

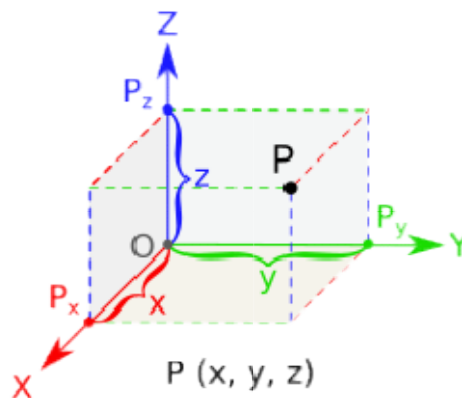
Рачунање перспективне пројекције се ради уз помоћ ове формуле:

$$x' = \frac{x * f}{z}, y' = \frac{y * f}{z} \quad f \rightarrow \text{жижна даљина}$$

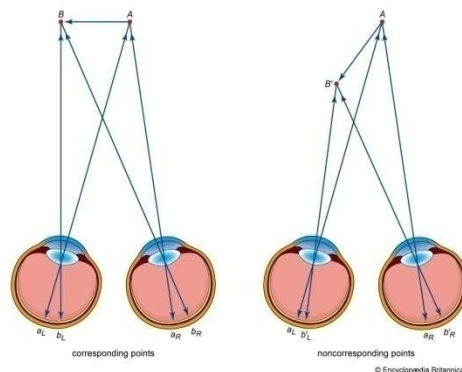
За проналажење тачака у односу на камеру, поштујући перспективу користимо матрикс:

$$\begin{bmatrix} d_x \\ d_y \\ d_z \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta_x) & \sin(\theta_x) \\ 0 & -\sin(\theta_x) & \cos(\theta_x) \end{bmatrix} \begin{bmatrix} \cos(\theta_y) & 0 & -\sin(\theta_y) \\ 0 & 1 & 0 \\ \sin(\theta_y) & 0 & \cos(\theta_y) \end{bmatrix} \begin{bmatrix} \cos(\theta_z) & \sin(\theta_z) & 0 \\ -\sin(\theta_z) & \cos(\theta_z) & 0 \\ 0 & 0 & 1 \end{bmatrix} \left(\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} - \begin{bmatrix} c_x \\ c_y \\ c_z \end{bmatrix} \right)$$

Перспективна пројекција захтева детаљну дефиницију од ортографске јер имамо видно поље, позицију и ротацију камере кроз коју гледамо у објекте. Кључне варијабле неопходне за рачунање матрикса су позиције тачака **A**, камеру **C** и оријентацију камере и позицију површине екрана на односу на кој се пројектују тачке.



Слика 2- 3Д простор; Тачка у 3Д простору;
Figure 2 – 3D Space; Point in 3D space;



Слика 3- Перцепција перспективе људи;
Figure 3 – Perception of perspective in humans;

⁵ то јест'

⁶ енг. 3D Render Engine, други назив за софтвер за 3Д манипулацију или симулацију

С обзиром да пројектујемо из 3Д-а у 2Д прво одређујемо позицију и ротацију камере, и на основу $pos.^7$ и $rot.^8$ камере као и узимање у обзир видно поље које ми произвољно стављамо (у случају Вајрцена користимо 70° ; стандард који се користи јер је то и степен видног поља људи) пројектујемо тачке А из 3Д у 2Д преко видног поља камере на екран.

Сада можемо да кренемо са тумачењем рада апликације!

Пре свега, да бисмо ефективно могли да пронађемо објекте који се налазе у једној сцени⁹ и да мењамо информације у њима направио сам нови struct¹⁰ који сам назвао “ObjectParams” (Параметри објекта).

У њега описујемо име датог објекта, тачке тог објекта, позицију у односу на свет^{**}, локална ротација и конекције између тачака у којој се уносе тачке које се повезују и боја те линије.

```
19 references
public struct ObjectParams()
{
    12 references
    public string? ObjectName { get; set; }
    5 references
    public Point3D[]? Points { get; set; }
    6 references
    public Point3D LocalRot { get; set; }
    6 references
    public Point3D WorldPos { get; set; }
    2 references
    public List<(int, int, Color)? ObjectConnections { get; set; }

    2 references
    public ObjectParams(string ObjN, Point3D[] Pts, Point3D Rot, Point3D Pos, List<(int, int, Color)> OC)
        : this()
    {
        ObjectName = ObjN;
        Points = Pts;
        LocalRot = Rot;
        WorldPos = Pos;
        ObjectConnections = OC;
    }
}
```

Слика 4- ObjectParams структура;
Figure 4 – ObjectParams structure;

Друга структура под именом “Point3D” јесте начин на који одређујемо позиције свих параметара којима је неопходно проналажење њихове позиције у 3Д свету.

```
82 references
public struct Point3D
{
    20 references
    public float X { get; set; }
    20 references
    public float Y { get; set; }
    20 references
    public float Z { get; set; }

    44 references
    public Point3D(float x, float y, float z)
    {
        X = x;
        Y = y;
        Z = z;
    }
}
```

Слика 5 – Point3D структура;
Figure 5 – Point3D structure;

⁷ позиција (камере)

⁸ ротација/оријентација (камере)

⁹ енг. scene – представља једну “сцену” у којој се налазе ти објекти

¹⁰ енг. structure – је структура која обухвата и чува податке и сродне функције

^{**} позиција од центра света, стављено је као “WorldPos” јер постоје и позиције тачака које су на релативној позицији од центра објекта који узима исту ту (WorldPos) позицију као њихова почетна позиција

```
List<ObjectParams> objects = new List<ObjectParams>();
public bool DrawObjectNums = false;
```

Слика 6 – Листа објеката;
Figure 6 – List of objects;

```
Point3D[] KockaPoints = new Point3D[]
{
    new Point3D(-50, -50, -50), 0
    new Point3D(50, -50, -50), 1
    new Point3D(50, 50, -50), 2
    new Point3D(-50, 50, -50), 3
    new Point3D(-50, -50, 50), 4
    new Point3D(50, -50, 50), 5
    new Point3D(50, 50, 50), 6
    new Point3D(-50, 50, 50), 7
};

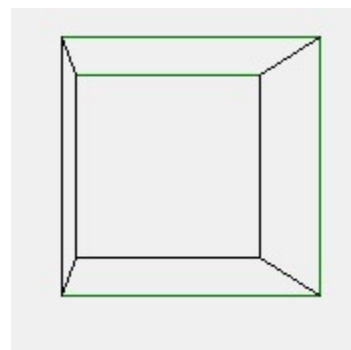
List<(int, int, Color)> KockaConnections = new List<(int, int, Color)>
{
    (0, 1, Color.Green), (1, 2, Color.Green), (2, 3, Color.Green), (3, 0, Color.Black), // bottom
    (4, 5, Color.Black), (5, 6, Color.Black), (6, 7, Color.Green), (7, 4, Color.Black), // top
    (0, 4, Color.Black), (1, 5, Color.Black), (2, 6, Color.Black), (3, 7, Color.Black) // sides
};

objects.Add(new ObjectParams("Cube", KockaPoints, new Point3D(0, 0, 0), new Point3D(30, 0, 0), KockaConnections));
```

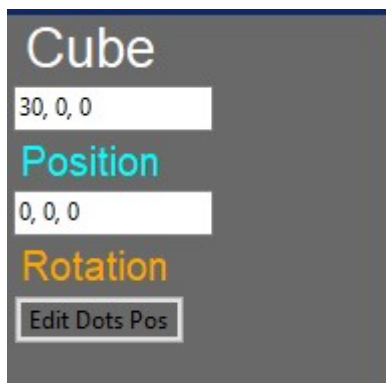
Слика 7 – Креирање објекта "Cube";
Figure 7 – Creating an object called "Cube";

Креирањем ових тачака и повезивањем истих правимо нови објекат под именом "Cube". Након можемо да подесимо ротацију и позицију објекта.

Али не морамо да излазимо из апликације да бисмо подешавали објекат. Са стране постоји "Dock" тј. алатна трака уз помоћ које можете да подешавате објекте. Кликом на део екрана где се налазе објекти, "Scene Viewer", отвара се листа објеката у тој сцени. Кликом на њих отвара се едитор¹¹ и можете мењати позицију, ротацију, позиције тачака и конекције.

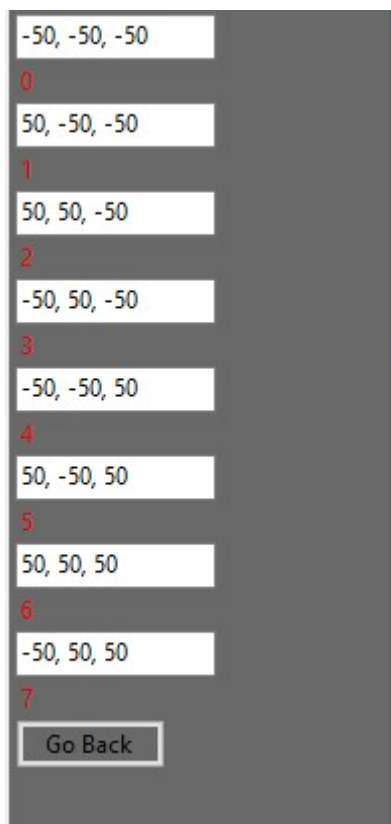


Слика 8 – Изглед објекта "Cube";
Figure 8 – How the object "Cube" looks;

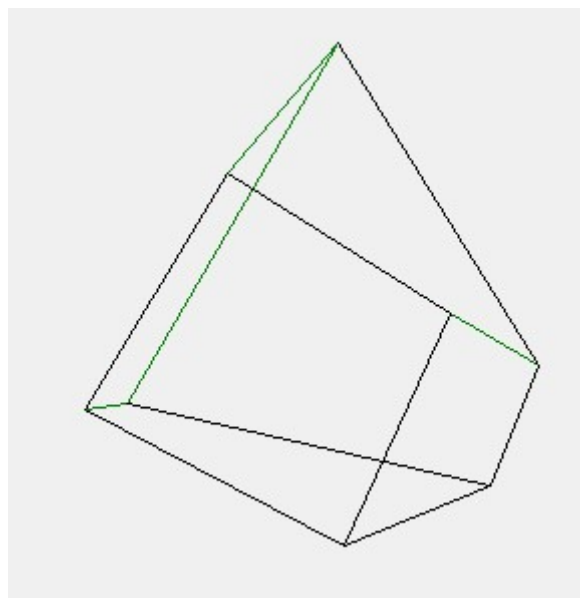


Слика 9 – Едитор слика 1;
Figure 9 – Editor Figure 1;

¹¹ уређивач



Слика 10 – Едитор слика 2;
Figure 10 – Editor Figure 2;



Слика 11 – Објект након промена;
Figure 11 – Cube after the iterations;

Како смо ово урадили?

```
3 references
class SelectItemToPanel
{
    private Form1 formInstance;

    1 reference
    public Form1 Form1 { get; }
    2 references
    public FlowLayoutPanel FlowLayoutPanel1 { get; set; }
    2 references
    public List<ObjectParams> objects { get; set; }
    1 reference
    public bool DrawObjectNums { get; set; }
    2 references
    public PictureBox PB { get; set; }

    1 reference
    public SelectItemToPanel(Form1 form, FlowLayoutPanel panel, List<ObjectParams> objs, bool DON, PictureBox PBox)
    {
```

Слика 12 – Класа SelectItemToPanel;
Figure 12 – Class SelectItemToPanel;

Правимо нову класу уз помоћ које можемо да правимо нове функције. “SelectItemToPanel” функција проналази све објекте у листи “objects” и прави нове дугмиће за њих и убацује их у уређивач.


```

public class ChangeProperty
{
    1 reference
    public void ShowBasicSettings(List<ObjectParams> objects, string clickedButton, FlowLayoutPanel FlowLayoutPanel1, PictureBox PB)...
    1 reference
    public void EditDotPositions(List<ObjectParams> Parameters, string ObjectName, FlowLayoutPanel Panel, PictureBox PB)...
    1 reference
    public void EditDotPositionsChanged(string NewCoords, List<ObjectParams> Objs, string ObjectName, string TBName, PictureBox PictureBox)...
    1 reference
    public void EditFullObjectPosition(string NewCoords, List<ObjectParams> Objs, string Obn, PictureBox PictureBox)...
    1 reference
    public void EditFullObjectRotation(string NewCoords, List<ObjectParams> Objs, string Obn, PictureBox PictureBox)...
    0 references
    public void EditConnections(List<ObjectParams> Parameters, string ObjectName, FlowLayoutPanel Panel, PictureBox PB)
}

```

Слика 13 – Класа ChangeProperty;
Figure 13 – Class ChangeProperty;

Након клика на неку од дугмади, позива се једна од офих функција класе “ChangeProperty” који контролише и мења својства објета. Приликом клика приказује се “ShowBasicSettings” који чисти уређивач и убацује нове објекте. Након клика на дугме “Edit Dots Pos”(страница 5,

```

public void EditDotPositions(List<ObjectParams> Parameters, string ObjectName, FlowLayoutPanel Panel, PictureBox PB)
{
    Panel.Controls.Clear();
    int index = 0;
    ObjectParams? selectedObj = Parameters.FirstOrDefault(obj => obj.ObjectName == ObjectName);

    if (selectedObj != null)
    {
        foreach (Point3D point in selectedObj.Value.Points)
        {
            TextBox tb = new TextBox()
            {
                Name = index.ToString(),
                Text = $"{point.X}, {point.Y}, {point.Z}",
                AutoSize = true
            };
            Label l = new Label()
            {
                Location = new Point(tb.Location.X + 30, tb.Location.Y),
                Text = index.ToString(),
                ForeColor = Color.Red,
                AutoSize = true
            };
            Panel.Controls.Add(tb); Panel.Controls.Add(l);
            index += 1;
            tb.TextChanged += (sender, e) => EditDotPositionsChanged(tb.Text, Parameters, ObjectName, tb.Name, PB);
        }
        Button BackBtn = new Button() { Name = "BackBtn", AutoSize = true, Text = "Go Back" };
        Panel.Controls.Add(BackBtn);
    }
}

```

Слика 14 – Функција EditDotPositions;
Figure 14 – Function EditDotPositions;

```

public void EditDotPositionsChanged(string NewCoords, List<ObjectParams> Objs, string ObjectName, string TBName, PictureBox PictureBox)
{
    try
    {
        float[] result = ParseStringToFloats(NewCoords);

        ObjectParams? selectedObj = Objs.FirstOrDefault(obj => obj.ObjectName == ObjectName);
        if (selectedObj.HasValue)
        {
            if (int.TryParse(TBName, out int index) && index >= 0 && index < selectedObj.Value.Points.Length)
            {
                selectedObj.Value.Points[index] = new Point3D(result[0], result[1], result[2]);
                PictureBox.Invalidate();
            }
            else
            {
                Console.WriteLine($"Invalid index: {TBName}");
            }
        }
        else
        {
            Console.WriteLine($"Couldn't find object named " + ObjectName);
        }
    }
    catch (FormatException ex)
    {
        Console.WriteLine($"Input format error: {ex.Message}");
    }
    catch (ArgumentException ex)
    {
    }
}

```

Слика 15 – Функција EditDotPositionChanged;
Figure 15 – Function EditDotPositionChanged;

слика 9) проверава се да ли објекат постоји и ако постоји појављују се информације о тачкама (страница 6, слика 10). Овај код гледа ако се промени један од тексбокова који су направљени и шаље их функцији “EditDotPositionsChanged” који контролише и мења позиције тачака.

Функција гледа да ли објекат који се шаље постоји и ако је формат у коме је написан текст у исправном формату уз помоћ функције “ParseStringToFloats”.

Овај део кода гледа да ли је формат добар и парсује га тако да нема ни један размак између координата, (формат изгледа овако: 0, 0, 0 – x, y, z) одваја их на 3 дела и шаље га назад.

Исто ради и за промене ротације и позиције целог објекта.

```
3 references
public static float[] ParseStringToFloats(string input)
{
    if (string.IsNullOrEmpty(input))
    {
        throw new ArgumentException("Input string is null or empty.");
    }
    string normalizedInput = Regex.Replace(input, @"\s+", "");
    string[] parts = normalizedInput.Split(new char[] { ',' }, StringSplitOptions.RemoveEmptyEntries);
    if (parts.Length != 3)
    {
        throw new FormatException($"Expected 3 numbers, but found {parts.Length}.");
    }
    float[] numbers = new float[3];
    for (int i = 0; i < parts.Length; i++)
    {
        if (!float.TryParse(parts[i], NumberStyles.Float, CultureInfo.InvariantCulture, out numbers[i]))
        {
            throw new FormatException($"Unable to parse '{parts[i]}' as a float.");
        }
    }
    return numbers;
}
```

Слика 16 – Функција ParseStringToFloats;
Figure 16 – Function ParseStringToFloats;

```
1 reference
public Point[] Draw(Point3D[] Points, Camera camera, PictureBox pictureBox, List<int, int, Color> ObjectConnections, PaintEventArgs e, Point3D Pos, Point3D Rot, bool DrawObjectNums)
{
    List<Point> projectedPoints = new List<Point>();
    float centerX = pictureBox.Width / 2;
    float centerY = pictureBox.Height / 2;
    DrawLines drawLines = new DrawLines();
    Point3D center = new Point3D(0, 0, 0);

    float fov = 70f;
    float aspectRatio = 0.6f;
    float fovRad = 1f / ((float)Math.Tan(fov * 0.5f * Math.PI / 180f));

    foreach (var point in Points)
    {
        float x = point.X - center.X + Pos.X;
        float y = point.Y - center.Y + Pos.Y;
        float z = point.Z - center.Z + Pos.Z;

        float tempY = y;
        y = (float)(tempY * Math.Cos(Rot.X * Math.PI / 180) - z * Math.Sin(Rot.X * Math.PI / 180));
        z = (float)(tempY * Math.Sin(Rot.X * Math.PI / 180) + z * Math.Cos(Rot.X * Math.PI / 180));

        float tempX = x;
        x = (float)(x * Math.Cos(Rot.Y * Math.PI / 180) + z * Math.Sin(Rot.Y * Math.PI / 180));
        z = (float)(-tempX * Math.Sin(Rot.Y * Math.PI / 180) + z * Math.Cos(Rot.Y * Math.PI / 180));

        float tempX2 = x;
        float tempY2 = y;
        x = (float)(tempX2 * Math.Cos(Rot.Z * Math.PI / 180) - tempY2 * Math.Sin(Rot.Z * Math.PI / 180));
        y = (float)(tempX2 * Math.Sin(Rot.Z * Math.PI / 180) + tempY2 * Math.Cos(Rot.Z * Math.PI / 180));

        x -= camera.Position.X;
        y -= camera.Position.Y;
        z -= camera.Position.Z;

        if (z <= 0.1f) z = 0.1f;

        float projectedX = (x * fovRad * aspectRatio) / z;
        float projectedY = (y * fovRad) / z;

        int screenX = (int)(centerX + projectedX + centerX);
        int screenY = (int)(centerY - projectedY + centerY);

        projectedPoints.Add(new Point(screenX, screenY));
    }

    foreach (var connection in ObjectConnections)
    {
        Point screenPoint1 = projectedPoints[connection.Item1];
        Point screenPoint2 = projectedPoints[connection.Item2];

        Pen pen = new Pen(connection.Item3);
        e.Graphics.DrawLine(pen, screenPoint1, screenPoint2);
    }

    if (DrawObjectNums == true)
        DrawNumbers(projectedPoints.ToArray(), e, new Font("Arial", 12), Brushes.Red);
    DrawAxesOverlay(e, camera, pictureBox);

    return projectedPoints.ToArray();
}
```

Функција “Draw” у класи “DrawLines” заправо рачуна и црта линије калкулишући изобличење од стране камере, ротацију, позицију објекта и камере. Након тога се цртају све тачке. “DrawAxisOverlay” јесте гизмо¹² који се види доле-лево (страна 1, слика 1).

```
foreach (var point in Points)
{
    float x = point.X - center.X + Pos.X;
    float y = point.Y - center.Y + Pos.Y;
    float z = point.Z - center.Z + Pos.Z;

    float tempY = y;
    y = (float)(tempY * Math.Cos(Rot.X * Math.PI / 180) - z * Math.Sin(Rot.X * Math.PI / 180));
    z = (float)(tempY * Math.Sin(Rot.X * Math.PI / 180) + z * Math.Cos(Rot.X * Math.PI / 180));

    float tempX = x;
    x = (float)(x * Math.Cos(Rot.Y * Math.PI / 180) + z * Math.Sin(Rot.Y * Math.PI / 180));
    z = (float)(-tempX * Math.Sin(Rot.Y * Math.PI / 180) + z * Math.Cos(Rot.Y * Math.PI / 180));

    float tempX2 = x;
    float tempY2 = y;
    x = (float)(tempX2 * Math.Cos(Rot.Z * Math.PI / 180) - tempY2 * Math.Sin(Rot.Z * Math.PI / 180));
    y = (float)(tempX2 * Math.Sin(Rot.Z * Math.PI / 180) + tempY2 * Math.Cos(Rot.Z * Math.PI / 180));

    x -= camera.Position.X;
    y -= camera.Position.Y;
    z -= camera.Position.Z;

    if (z <= 0.1f) z = 0.1f;

    float projectedX = (x * fovRad * aspectRatio) / z;
    float projectedY = (y * fovRad) / z;

    int screenX = (int)(centerX + projectedX + centerX);
    int screenY = (int)(centerY - projectedY + centerY);

    projectedPoints.Add(new Point(screenX, screenY));
}
```

```
foreach (var connection in ObjectConnections)
{
    Point screenPoint1 = projectedPoints[connection.Item1];
    Point screenPoint2 = projectedPoints[connection.Item2];

    Pen pen = new Pen(connection.Item3);
    e.Graphics.DrawLine(pen, screenPoint1, screenPoint2);
}

if (DrawObjectNums == true)
    DrawNumbers(projectedPoints.ToArray(), e, new Font("Arial", 12), Brushes.Red);
DrawAxesOverlay(e, camera, pictureBox);

return projectedPoints.ToArray();
```

Слика 17,18 и 19 – Функција Draw, рачунање линија, исцртавање линија;
Figure 17, 18 and 19 – Function Draw, calculating the lines, drawing the lines;

¹² приказ оса у 3Д рендер софтверима


```
private void Form1_KeyDown(object sender, KeyEventArgs e)
{
    float panSpeed = 10f;

    if (e.KeyCode == Keys.W)
    {
        Point3D forward = camera.GetForwardDirection();
        camera.Position += forward * panSpeed;
        pictureBox1.Invalidate();
    }

    if (e.KeyCode == Keys.S)
    {
        Point3D forward = camera.GetForwardDirection();
        camera.Position -= forward * panSpeed;
        pictureBox1.Invalidate();
    }

    if (e.KeyCode == Keys.A)
    {
        Point3D right = camera.GetRightDirection();
        camera.Position -= right * panSpeed;
        pictureBox1.Invalidate();
    }
}
```

```
private void pictureBox1_MouseDown(object sender, MouseEventArgs e)
{
    if (isMouseDownRight)
    {
        float sensitivity = 0.005f;
        int dx = e.X - lastMousePosition.X;
        int dy = e.Y - lastMousePosition.Y;

        camera.UpdateCameraOrientation(dx, dy, sensitivity);

        lastMousePosition = e.Location;

        pictureBox1.Invalidate();
    }
}
```

Овај део кода помера/ротира камеру.

РЕЗУЛТАТИ ИСТРАЖИВАЊА И ДИСКУСИЈА

Током истраживања подценио сам оно што треба да урадим за реализацију овако опширног рада и наравно има још доста ствари које могу да додам. Схватио сам колико је неопходно да код буде компјутеру лак за разумевање ако хоћемо да имамо програм који могу већина, па чак и сви рачунари да без обструкција га користе; као и приказивање илузије 3Д света иако га ми видимо у 2Д-у.

То су неки од фактора којима се ми служимо у компјутерској графици за управо овакве софтвере.

Овакви програми су нам све више и више неопходни како због игрица, тако и за уметнике који користе компјутере за прављење дигиталних цртежа и архитекте и инжењере који праве брло комплексне моделе који морају да прате физику.

Сматрам да и мој пројекат може да приближи људима 3Д софтвере на један врло лак начин.

ЗАКЉУЧАК

Радећи на овом пројекту дошао сам на доста препрека приликом реализовања. Било је доста терминологије које никад не бих знао да нисам радио на овом пројекту и уједно ме је направило бољим програмером.

Сада познајем доста о томе како раде компјутерски генерисани 3Д објекти, како функционише наш вид и како можемо да га симулишемо унутар компјутера и како раде 3Д рендер софтвери. Уједно као пројекат који сам хтео да направим још од почетка школовања у центру, ово је испуњење мојег вишегодишњег сна.

У будућности планирам да још надограђујем овај пројекат јер има доста потенцијала, као што су шејдери, текстуре, осветљење, могућност пребацивања у .obj фајлове и сл. Ово је пројекат који може да донесе доста бенефита људима и мислим да континуираним радом на овој апликацији као и избацивање Вајрцена у свет је у блиској будућности.

ЗАХВАЛНИЦА

- Захваљујем се мојим менторима за смернице и идеје током реализације овог пројекта,
- Захваљујем се Центру за таленте Земун – 1 за могућност да се бавим оним што ме занима као и могућност да се такмичим на овако високом нивоу.

ЛИТЕРАТУРА

- Јутјуб видео; “Ротације у 3Д-у”: <https://www.youtube.com/watch?v=wg9bI8-Qx2Q>
- Стековерфлоу; пројекција 3Д тачака у 2Д-у: <https://stackoverflow.com/questions/23472048/projecting-3d-points-to-2d-plane>
- Гугл учионица: <https://classroom.google.com/>
- Визуал Студио документација: <https://learn.microsoft.com/en-us/visualstudio/windows/?view=vs-2022>
- Википедија, тродимензионални простор: https://en.wikipedia.org/wiki/Three-dimensional_space
- Википедија, пројекције: https://en.wikipedia.org/wiki/3D_projection
- Википедија, Еулерови углови: https://en.wikipedia.org/wiki/Euler_angles
- Scratchpixel, перспективска и ортографска пројекција; матрикси: <https://www.scratchapixel.com/lessons/3d-basic-rendering/perspective-and-orthographic-projection-matrix/building-basic-perspective-projection-matrix.html>